UNIVA

Universidad del Valle de Atemajac

Programación Orientada a Objetos

Práctica 1

Sintaxis

Alumno: Tiberio Velasco Lozano

Matrícula: 0126310099

Docente: Irving Edgar Mendoza Ayala

Fecha: 29 de septiembre de 2026

Introducción

La Programación Orientada a Objetos (POO) organiza un programa a partir de objetos que representan elementos del mundo real. Cada objeto se crea a partir de una clase, que funciona como una plantilla: define sus características (atributos) y las acciones que puede realizar (métodos).

Dos principios de la POO guían esta práctica. La abstracción consiste en tomar solo las características importantes de un objeto real e ignorar las demás; por ejemplo, de un animal nos interesan su especie, su peso o su hábitat. La herencia permite que una clase hija reciba los atributos y métodos de una clase padre, de modo que el código común se escribe una sola vez.

En esta práctica se define la clase base Animal dentro del paquete SeresVivos, y a partir de ella las subclases AnimalDomestico y AnimalGranja. Cada clase se describe primero con la sintaxis de nomenclatura vista en clase (nombre, atributos y métodos) y después se implementa en Java.

Desarrollo de la actividad

1. Sintaxis utilizada

Nombre de la clase

Se escribe como Paquete :: NombreDeLaClase. Ejemplo: SeresVivos :: Animal.

Atributos

```
[visibilidad] nombre [multiplicidad] [: tipo] [= valor inicial] [{lista-
                                propiedades}]
```

Elemento	Significado
+ pública	El atributo es visible para todos los clientes de la clase.
- privada	El atributo solo es visible dentro de la propia clase.
# protegida	El atributo es visible para las subclases de la clase.
tipo	Tipo de dato del atributo (int, double, String, boolean…).
multiplicidad	Cuántos valores guarda el atributo. Si no se escribe vale 1; [0..1] indica que puede no tener valor, [1..3] de uno a tres y [0..*] cualquier cantidad.
valor inicial	Valor que tiene el atributo cuando se crea el objeto.
lista-propiedades	Valores permitidos para el atributo; sirve para tipos enumerados, por ejemplo {MACHO, HEMBRA}.
ámbito de clase	Atributo o método compartido por todos los objetos de la clase (static). Se representa subrayado.

Métodos

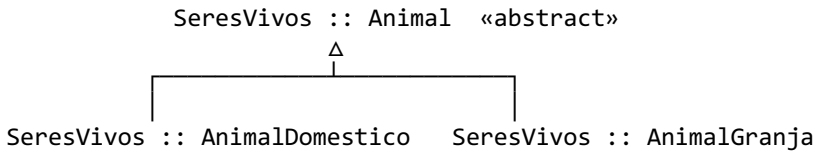
```
[visibilidad] nombre [(lista de parámetros)] [: tipo devuelto] [{lista-
                                propiedades}]
```

Cada parámetro se escribe como nombre : tipo. En la lista de propiedades se indica, por ejemplo, {abstract} cuando el método no tiene cuerpo y cada subclase debe implementarlo, o {redefinido} cuando la subclase sobrescribe un método heredado.

2. Abstracción y herencia

Del mundo real se tomaron las características que comparten todos los animales: número de patas, número de ojos, especie, peso, color, tipo de pelaje, región, hábitat, género y expectativa de vida. Estas forman la clase base Animal. Como no existe un "animal" genérico, Animal se declara abstracta: no se crean objetos de ella, solo de sus subclases.

AnimalDomestico y AnimalGranja heredan de Animal (extends), así que ya cuentan con esos diez atributos y con los métodos comer(), dormir() y mostrarDatos(). Cada subclase agrega solo lo que la distingue.



3. Clase base: SeresVivos :: Animal

«abstract» ***SeresVivos :: Animal***
numPatas : int = 4 # numOjos : int = 2 # especie : String # peso : double = 0.0 # color [1..3] : String # tipoPelaje : TipoPelaje {CORTO, LARGO, PLUMAS, LANA, SIN_PELO} # region : String # habitat : String # genero : Genero {MACHO, HEMBRA} # expectativaVida : int + totalAnimales : int = 0
+ Animal(especie : String, numPatas : int, peso : double, color :

String[], tipoPelaje : TipoPelaje, region : String, habitat : String, genero : Genero, expectativaVida : int) + hacerSonido() : String {abstract} + comer(kilos : double) : void + dormir(horas : int) : void + mostrarDatos() : void + getTotalAnimales() : int + getEspecie() : String + getPeso() : double + setPeso(peso : double) : void

Los atributos son protegidos (#) para que las subclases puedan usarlos directamente. color tiene multiplicidad [1..3] porque un animal puede tener hasta tres colores. tipoPelaje y genero usan una lista de propiedades, que en Java se implementa con enum. totalAnimales y getTotalAnimales() tienen ámbito de clase (static): cuentan cuántos animales se han creado en total.

4. Subclase: SeresVivos :: AnimalDomestico

SeresVivos :: AnimalDomestico extends Animal
- nombre : String - raza : String - dueno : String - entrenado : boolean = false - vacunas [0..*] : String
+ AnimalDomestico(nombre : String, raza : String, dueno : String, especie : String, peso : double, color : String[], tipoPelaje : TipoPelaje, genero : Genero, expectativaVida : int) + hacerSonido() : String {redefinido} + jugar(juguete : String) : void + vacunar(vacuna : String) : void + entrenar(truco : String) : void + mostrarDatos() : void {redefinido} + getNombre() : String + setDueno(dueno : String) : void

Agrega cinco atributos privados (-) propios de una mascota. vacunas tiene multiplicidad [0..*] porque la mascota puede no tener vacunas o tener varias. Sus métodos propios son jugar(), vacunar() y entrenar(); además implementa hacerSonido() según la especie (perro o gato).

5. Subclase: SeresVivos :: AnimalGranja

SeresVivos :: AnimalGranja extends Animal
- idArete : String - producto : Producto {LECHE, HUEVO, LANA, CARNE} - produccionDiaria : double = 0.0 - ubicacion : String = "Corral 1" - fechaIngreso : String
+ AnimalGranja(idArete : String, producto : Producto, fechaIngreso : String, especie : String, numPatas : int, peso : double, color : String[], tipoPelaje : TipoPelaje, genero : Genero, expectativaVida : int) + hacerSonido() : String {redefinido} + producir() : double + trasladar(nuevaUbicacion : String) : void + revisarSalud() : void + mostrarDatos() : void {redefinido} + getProducto() : Producto + getUbicacion() : String

Agrega cinco atributos privados relacionados con el control de la granja. producto usa una lista de propiedades con los productos permitidos, y ubicacion inicia con el valor "Corral 1". Sus métodos propios son producir(), que regresa un double con la producción del día, trasladar() y revisarSalud().

6. Código fuente en Java

Las tres clases se guardaron en la carpeta SeresVivos, que corresponde al paquete (package SeresVivos;). El programa principal practica_1.java las importa y crea un objeto de cada subclase.

SeresVivos/Animal.java

```
package SeresVivos;

/*
 * SeresVivos :: Animal
 * Clase base (abstracta). Reune los atributos y metodos que comparten todos los animales.
 * No se pueden crear objetos de Animal directamente, solo de sus subclases.
 */
public abstract class Animal {

    // ---------------- LISTAS DE PROPIEDADES (tipos enumerados) ----------------
    // Solo se permiten estos valores en los atributos genero y tipoPelaje
    public enum Genero { MACHO, HEMBRA }
    public enum TipoPelaje { CORTO, LARGO, PLUMAS, LANA, SIN_PELO }

    // ---------------- ATRIBUTOS ----------------
    // Sintaxis: [visibilidad] nombre [multiplicidad] [:tipo] [= valor inicial] [lista-propiedades]
```

```
    // protected (#): visibles para las subclases AnimalDomestico y AnimalGranja
    protected int numPatas = 4;
    protected int numOjos = 2;
    protected String especie;
    protected double peso = 0.0;
    protected String[] color = new String[3];      // multiplicidad [1..3]
    protected TipoPelaje tipoPelaje;
    protected String region;
    protected String habitat;
    protected Genero genero;
    protected int expectativaVida;

    // Atributo con ambito de clase (static): lo comparten todos los objetos
    public static int totalAnimales = 0;

    // ---------------- CONSTRUCTOR ----------------
    public Animal(String especie, int numPatas, double peso, String[] color, TipoPelaje tipoPelaje,
                  String region, String habitat, Genero genero, int expectativaVida) {
        this.especie = especie;
        this.numPatas = numPatas;
        this.peso = peso;
        this.color = color;
        this.tipoPelaje = tipoPelaje;
        this.region = region;
        this.habitat = habitat;
        this.genero = genero;
        this.expectativaVida = expectativaVida;
        totalAnimales++;
    }

    // ---------------- METODOS ----------------
    // Sintaxis: [visibilidad] nombre [(lista de parametros)] [:tipo devuelto] [lista-propiedades]

    // Metodo abstracto: cada subclase decide que sonido hace su animal
    public abstract String hacerSonido();

    // Aumenta el peso segun los kilos que come (10% se convierte en peso)
    public void comer(double kilos) {
        peso += kilos * 0.1;
        System.out.println("El/la " + especie + " comio " + kilos + " kg. Peso actual: "
                + String.format("%.2f", peso) + " kg");
    }

    public void dormir(int horas) {
        System.out.println("El/la " + especie + " durmio " + horas + " horas en su " + habitat + ".");
    }

    // Imprime los atributos comunes; las subclases lo amplian con los suyos
    public void mostrarDatos() {
        System.out.println("Especie: " + especie + " | Genero: " + genero);
        System.out.println("Patas: " + numPatas + " | Ojos: " + numOjos + " | Peso: "
                + String.format("%.2f", peso) + " kg");
        System.out.println("Color: " + String.join(", ", color) + " | Pelaje: " + tipoPelaje);
        System.out.println("Region: " + region + " | Habitat: " + habitat);
        System.out.println("Expectativa de vida: " + expectativaVida + " años");
    }

    // Metodo de clase (static): se llama con Animal.getTotalAnimales()
    public static int getTotalAnimales() {
        return totalAnimales;
    }

    // ---------------- GETTERS Y SETTERS ----------------
    public String getEspecie() {
        return especie;
```

```
    }

    public double getPeso() {
        return peso;
    }

    public void setPeso(double peso) {
        this.peso = peso;
    }
}
```

SeresVivos/AnimalDomestico.java

```
package SeresVivos;

import java.util.ArrayList;

/*
 * SeresVivos :: AnimalDomestico
 * Hereda (extends) todos los atributos y metodos de Animal y agrega los suyos.
 */
public class AnimalDomestico extends Animal {

    // ---------------- ATRIBUTOS ----------------
    private String nombre;
    private String raza;
    private String dueno;
    private boolean entrenado = false;
    private ArrayList<String> vacunas = new ArrayList<>();   // multiplicidad [0..*]

    // ---------------- CONSTRUCTOR ----------------
    // super(...) manda los datos comunes al constructor de la clase base Animal
    public AnimalDomestico(String nombre, String raza, String dueno, String especie, double peso,
                           String[] color, TipoPelaje tipoPelaje, Genero genero, int expectativaVida) {
        super(especie, 4, peso, color, tipoPelaje, "Urbana", "Casa", genero, expectativaVida);
        this.nombre = nombre;
        this.raza = raza;
        this.dueno = dueno;
    }

    // ---------------- METODOS ----------------

    // Implementacion del metodo abstracto de Animal
    @Override
    public String hacerSonido() {
        if (especie.equalsIgnoreCase("perro")) {
            return nombre + " dice: ¡Guau guau!";
        } else if (especie.equalsIgnoreCase("gato")) {
            return nombre + " dice: ¡Miau!";
        }
        return nombre + " hace un sonido.";
    }

    public void jugar(String juguete) {
        System.out.println(nombre + " esta jugando con su " + juguete + ".");
    }

    public void vacunar(String vacuna) {
        vacunas.add(vacuna);
        System.out.println(nombre + " recibio la vacuna: " + vacuna + ".");
    }
```

```
    public void entrenar(String truco) {
        entrenado = true;
        System.out.println(nombre + " aprendio el truco: " + truco + ".");
    }

    // Sobrescribe mostrarDatos(): usa el de Animal y agrega los atributos propios
    @Override
    public void mostrarDatos() {
        System.out.println("Nombre: " + nombre + " | Raza: " + raza + " | Dueño: " + dueno);
        super.mostrarDatos();
        System.out.println("Entrenado: " + (entrenado ? "Si" : "No")
                + " | Vacunas: " + (vacunas.isEmpty() ? "Ninguna" : String.join(", ", vacunas)));
        System.out.println("---------------------------------------");
    }

    // ---------------- GETTERS Y SETTERS ----------------
    public String getNombre() {
        return nombre;
    }

    public void setDueno(String dueno) {
        this.dueno = dueno;
    }
}
```

SeresVivos/AnimalGranja.java

```
package SeresVivos;

/*
 * SeresVivos :: AnimalGranja
 * Hereda (extends) todos los atributos y metodos de Animal y agrega los suyos.
 */
public class AnimalGranja extends Animal {

    // Lista de propiedades: productos permitidos
    public enum Producto { LECHE, HUEVO, LANA, CARNE }

    // ---------------- ATRIBUTOS ----------------
    private String idArete;
    private Producto producto;
    private double produccionDiaria = 0.0;
    private String ubicacion = "Corral 1";
    private String fechaIngreso;

    // ---------------- CONSTRUCTOR ----------------
    public AnimalGranja(String idArete, Producto producto, String fechaIngreso, String especie, int
numPatas,
                        double peso, String[] color, TipoPelaje tipoPelaje, Genero genero, int
expectativaVida) {
        super(especie, numPatas, peso, color, tipoPelaje, "Rural", "Granja", genero, expectativaVida);
        this.idArete = idArete;
        this.producto = producto;
        this.fechaIngreso = fechaIngreso;
    }

    // ---------------- METODOS ----------------

    // Implementacion del metodo abstracto de Animal
    @Override
    public String hacerSonido() {
        if (especie.equalsIgnoreCase("vaca")) {
            return "La vaca " + idArete + " dice: ¡Muuu!";
```

```
        } else if (especie.equalsIgnoreCase("gallina")) {
            return "La gallina " + idArete + " dice: ¡Cocoroco!";
        }
        return "El animal " + idArete + " hace un sonido.";
    }

    // Regresa la produccion del dia (5% de su peso) y la guarda en el atributo
    public double producir() {
        produccionDiaria = peso * 0.05;
        return produccionDiaria;
    }

    public void trasladar(String nuevaUbicacion) {
        System.out.println(idArete + " se traslado de " + ubicacion + " a " + nuevaUbicacion + ".");
        ubicacion = nuevaUbicacion;
    }

    public void revisarSalud() {
        System.out.println("Revision de " + idArete + ": peso " + String.format("%.2f", peso) + " kg, "
                + (peso > 0 ? "estado saludable." : "requiere atencion."));
    }

    // Sobrescribe mostrarDatos(): usa el de Animal y agrega los atributos propios
    @Override
    public void mostrarDatos() {
        System.out.println("Arete: " + idArete + " | Ingreso: " + fechaIngreso + " | Ubicacion: " +
ubicacion);
        super.mostrarDatos();
        System.out.println("Producto: " + producto + " | Produccion diaria: "
                + String.format("%.2f", produccionDiaria));
        System.out.println("---------------------------------------");
    }

    // ---------------- GETTERS Y SETTERS ----------------
    public Producto getProducto() {
        return producto;
    }

    public String getUbicacion() {
        return ubicacion;
    }
}
```

practica_1.java

```
import SeresVivos.Animal;
import SeresVivos.AnimalDomestico;
import SeresVivos.AnimalGranja;

/*
 * Practica 1: Clases, Atributos y Metodos
 * Clase base: SeresVivos :: Animal
 * Subclases (herencia): SeresVivos :: AnimalDomestico y SeresVivos :: AnimalGranja
 *
 * Compilar: javac practica_1.java SeresVivos/*.java
 * Ejecutar: java practica_1
 */
public class practica_1 {

    public static void main(String[] args) {

        // ---------------- ANIMAL DOMESTICO ----------------
        AnimalDomestico mascota = new AnimalDomestico("Firulais", "Labrador", "Tiberio", "Perro", 25.0,
```

```
                new String[] {"Cafe", "Blanco"}, Animal.TipoPelaje.CORTO, Animal.Genero.MACHO, 12);

        System.out.println("\t\t*** ANIMAL DOMESTICO ***\n");
        mascota.mostrarDatos();
        System.out.println(mascota.hacerSonido());
        mascota.jugar("pelota");
        mascota.vacunar("Rabia");
        mascota.entrenar("sentarse");
        mascota.comer(2.0);          // metodo heredado de Animal
        mascota.mostrarDatos();

        // ---------------- ANIMAL DE GRANJA ----------------
        AnimalGranja vaca = new AnimalGranja("VAC-001", AnimalGranja.Producto.LECHE, "15/03/2024",
"Vaca", 4,
                450.0, new String[] {"Blanco", "Negro"}, Animal.TipoPelaje.CORTO, Animal.Genero.HEMBRA,
20);

        System.out.println("\n\t\t*** ANIMAL DE GRANJA ***\n");
        vaca.mostrarDatos();
        System.out.println(vaca.hacerSonido());
        vaca.comer(15.0);            // metodo heredado de Animal
        double litros = vaca.producir();
        System.out.println("Produccion del dia: " + String.format("%.2f", litros) + " litros de " +
vaca.getProducto());
        vaca.trasladar("Corral 2");
        vaca.dormir(6);              // metodo heredado de Animal
        vaca.revisarSalud();
        vaca.mostrarDatos();

        // Atributo de ambito de clase: se consulta desde la clase, no desde un objeto
        System.out.println("\nTotal de animales registrados: " + Animal.getTotalAnimales());
    }
}
```

7. Ejecución del programa

Se compiló con javac practica_1.java SeresVivos/*.java y se ejecutó con java practica_1:

Salida en consola

```
        *** ANIMAL DOMESTICO ***

Nombre: Firulais | Raza: Labrador | Dueño: Tiberio
Especie: Perro | Genero: MACHO
Patas: 4 | Ojos: 2 | Peso: 25.00 kg
Color: Cafe, Blanco | Pelaje: CORTO
Region: Urbana | Habitat: Casa
Expectativa de vida: 12 años
Entrenado: No | Vacunas: Ninguna
----------------------------------------
Firulais dice: ¡Guau guau!
Firulais esta jugando con su pelota.
Firulais recibio la vacuna: Rabia.
Firulais aprendio el truco: sentarse.
El/la Perro comio 2.0 kg. Peso actual: 25.20 kg
Nombre: Firulais | Raza: Labrador | Dueño: Tiberio
Especie: Perro | Genero: MACHO
Patas: 4 | Ojos: 2 | Peso: 25.20 kg
Color: Cafe, Blanco | Pelaje: CORTO
Region: Urbana | Habitat: Casa
```

```
Expectativa de vida: 12 años
Entrenado: Si | Vacunas: Rabia
--------------------------------------


        *** ANIMAL DE GRANJA ***


Arete: VAC-001 | Ingreso: 15/03/2024 | Ubicacion: Corral 1
Especie: Vaca | Genero: HEMBRA
Patas: 4 | Ojos: 2 | Peso: 450.00 kg
Color: Blanco, Negro | Pelaje: CORTO
Region: Rural | Habitat: Granja
Expectativa de vida: 20 años
Producto: LECHE | Produccion diaria: 0.00
--------------------------------------
La vaca VAC-001 dice: ¡Muuu!
El/la Vaca comio 15.0 kg. Peso actual: 451.50 kg
Produccion del dia: 22.58 litros de LECHE
VAC-001 se traslado de Corral 1 a Corral 2.
El/la Vaca durmio 6 horas en su Granja.
Revision de VAC-001: peso 451.50 kg, estado saludable.
Arete: VAC-001 | Ingreso: 15/03/2024 | Ubicacion: Corral 2
Especie: Vaca | Genero: HEMBRA
Patas: 4 | Ojos: 2 | Peso: 451.50 kg
Color: Blanco, Negro | Pelaje: CORTO
Region: Rural | Habitat: Granja
Expectativa de vida: 20 años
Producto: LECHE | Produccion diaria: 22.58
--------------------------------------


Total de animales registrados: 2
```

La salida muestra que las subclases usan los métodos heredados de Animal (comer() y dormir()) y también sus métodos propios, que cambian el estado del objeto: Firulais queda entrenado y con la vacuna de la rabia; la vaca aumenta de peso, registra su producción y pasa al Corral 2. Al final, el atributo de clase totalAnimales indica que se crearon 2 animales.

Conclusión

La práctica permitió aplicar la sintaxis completa para definir una clase: el nombre con su paquete (SeresVivos :: Animal), los atributos con su visibilidad, multiplicidad, tipo, valor inicial y lista de propiedades, y los métodos con sus parámetros y tipo devuelto. Escribir primero esta notación facilitó pasar después al código en Java, porque cada elemento tiene su equivalente directo: los símbolos +, - y # se convierten en public, private y protected; las listas de propiedades en enum; y el ámbito de clase en static.

Con la abstracción se identificaron las características comunes a todos los animales y se agruparon en la clase base Animal. Con la herencia, AnimalDomestico y

AnimalGranja reutilizaron esos atributos y métodos sin repetir código, y solo agregaron lo que las distingue. Así el programa queda más ordenado y es fácil de ampliar: para agregar, por ejemplo, un animal salvaje bastaría con crear otra subclase de Animal.